

FDE研修 未経験者コース ハンズオン教材(全7分野)

画面イメージと作業手順で学ぶ、16週間の実践テキスト

対象:未経験者コース受講者(プログラミング経験なし)/教材リポジトリ:company_inner_search_app2

関連資料:docs/fde-training-program.md(カリキュラム全体)、docs/fde-training-slides.html(研修スライド)

本書の使い方

- 本書は未経験者コース(16週間)の **7つの分野**を、章ごとに「学習ゴール → 作業手順 → 画面イメージ → 確認ポイント → つまずき対処」の順で解説します。
- 画面イメージ(▲マークの図)は、その手順を正しく実行できたときに見えるはずの画面です。自分の画面と見比べながら進めてください。
- コマンドは **1行ずつ自分の手で入力**してください(コピー&ペーストは、まず手で打って動いた後の2回目以降に)。
- 詰まったら **30分ルール**:30分自力で調べても解決しなければ、必ずメンターに質問しましょう。

第1～2週

①Python基礎

第3週

②Git/GitHub

第4週

③SQL

第5～6週

④Webアプリ

第7～10週

⑤生成AI・RAG

第11～13週

⑥コンサル基礎

第14～16週

⑦総合演習

目次

1. 第1章 Python基礎(第1～2週)

環境構築/演習の進め方/CSV集計課題

2. 第2章 Git・GitHub・ターミナル(第3週)

ブランチ→コミット→PR→レビューの一連の流れ

3. 第3章 SQLとデータモデリング(第4週)

サンプルDBへの問い合わせ演習

4. 第4章 Webアプリ基礎(第5～6週)

Streamlitで社内FAQアプリをゼロから作る

5. 第5章 生成AI・RAG入門(第7～10週)

OpenAI API/教材アプリの起動と改修

6. 第6章 コンサルティング基礎(第11～13週)

ロジックツリー/ヒアリング/要件定義書/プレゼン

7. 第7章 総合演習(第14～16週)

課題・スケジュール・成果物・評価基準

第1章 Python基礎

すべての土台。環境を整え、100本ノックで文法を体に覚えさせ、CSV集計スクリプトで「実データを扱う」最初の一步を踏み出します。

🎯 この章の学習ゴール

- 自分のPCに Python の開発環境を構築できる
- 変数・条件分岐・繰り返し・関数を使った 20〜30 行のプログラムを自力で書ける
- CSVファイルを読み込んで集計するスクリプトを完成させ、結果を説明できる

1-1. 開発環境の構築(所要:半日)

1 Python がインストールされているか確認する

ターミナル(Windows は「PowerShell」、Mac は「ターミナル」アプリ)を開き、次のコマンドを入力して Enter キーを押します。

```
python --version
```

※ Mac では `python3 --version` と入力してください。以降のコマンドも同様に読み替えます。



```
PowerShell — 研修用ターミナル

PS C:\Users\trainee> python --version
Python 3.11.9

PS C:\Users\trainee> _
```

▲ **画面イメージ:** バージョン番号(3.10以上ならOK)が表示されれば成功。「認識されません」と出たら Python 未インストールなので、python.org からインストールします(下のつまずき対処参照)。

2 研修用の作業フォルダと仮想環境を作る

デスクトップではなくホームフォルダ直下に `fde-training` フォルダを作り、その中に「仮想環境」を作ります。仮想環境とは、このプロジェクト専用の **Python 環境** のこと。他のプロジェクトとライブラリが混ざる事故を防げます。

```
mkdir fde-training
cd fde-training
python -m venv .venv
.venv\Scripts\activate      # Mac の場合: source .venv/bin/activate
```

PowerShell — 研修用ターミナル

```
PS C:\Users\trainee> mkdir fde-training
PS C:\Users\trainee> cd fde-training
PS C:\Users\trainee\fde-training> python -m venv .venv
PS C:\Users\trainee\fde-training> .venv\Scripts\activate
(.venv) PS C:\Users\trainee\fde-training> _
```

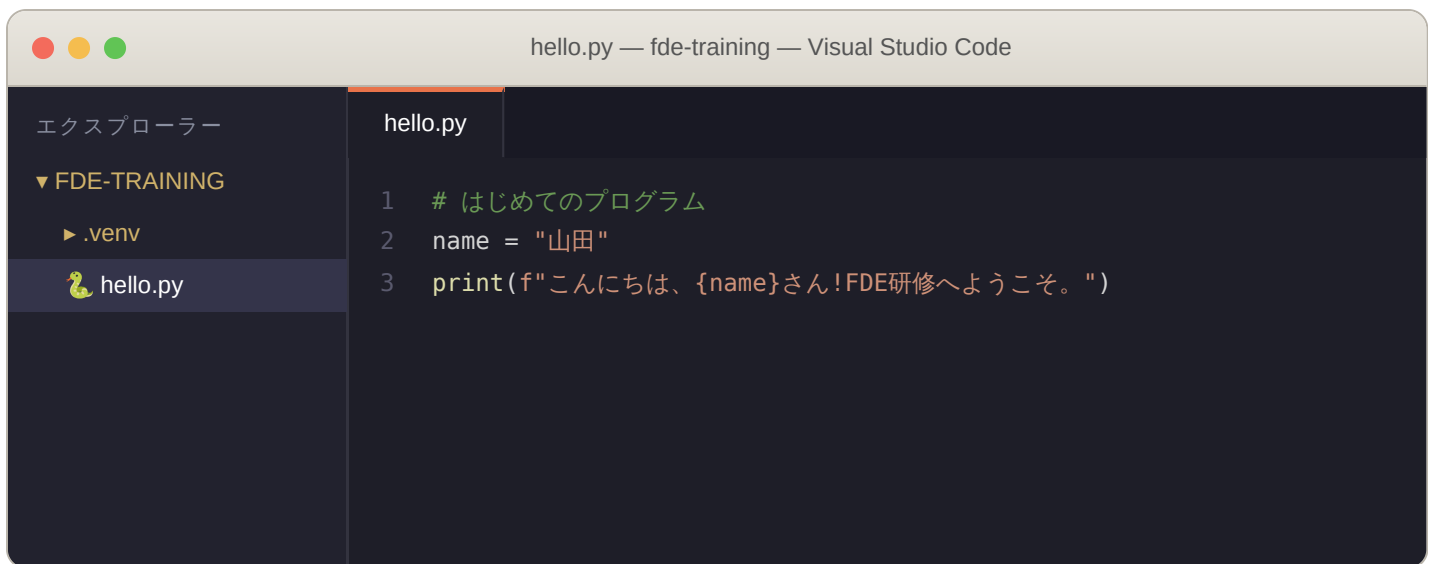
▲ **画面イメージ:** 行の先頭に `(.venv)` と表示されたら仮想環境が有効になっています。研修の作業は常にこの状態で行います。

3 エディタ(VS Code)を準備し、最初のプログラムを書く

VS Code(Visual Studio Code)をインストールし、拡張機能「Python」「Japanese Language Pack」を追加します。`fde-training` フォルダを VS Code で開き、`hello.py` という名前のファイルを作って次の3行を書きます。

hello.py

```
# はじめてのプログラム
name = "山田"
print(f"こんにちは、{name}さん!FDE研修へようこそ。")
```

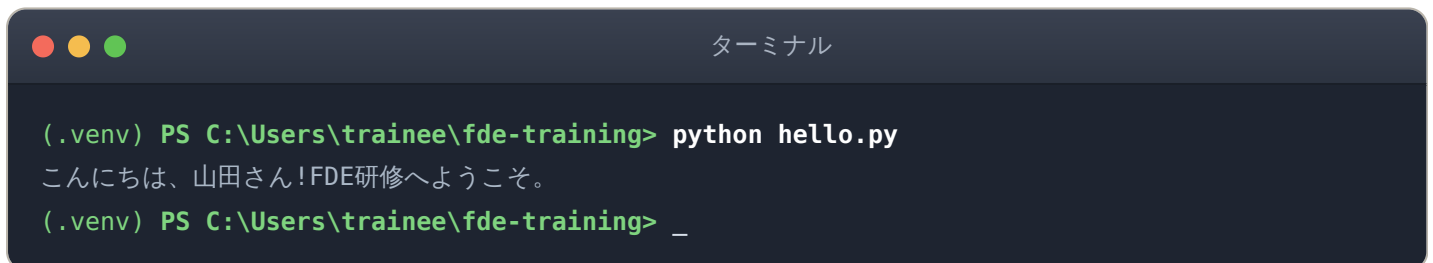


▲ **画面イメージ:** VS Code の画面。左が「エクスプローラー」(ファイル一覧)、右がエディタ。ファイル名の拡張子は必ず `.py` にします。

4 プログラムを実行する

ターミナルに戻り(VS Code 内蔵ターミナルでも可:メニュー「ターミナル → 新しいターミナル」)、実行します。

```
python hello.py
```



▲ **画面イメージ:** 自分の書いたプログラムが動いた瞬間。ここからすべてが始まります。

✓ 確認ポイント(1-1完了の条件)

- `python --version` で 3.10 以上が表示される
- プロンプトの先頭に `(.venv)` が付いた状態で作業できている
- `hello.py` の `name` を自分の名前に変えて再実行し、出力が変わることを確認した

1-2. 演習100本ノックの進め方(第1週後半～第2週前半)

1問5～15分の小さな問題を大量に解いて、文法を「思い出す」のではなく「手が勝手に動く」状態にします。問題は研修用リポジトリの `exercises/` から配布されます。進め方のルールは次の3つです。

1. **1問ごとに必ず実行する。**書いて満足せず、動かして結果を見る
2. **エラーは友達。**エラーメッセージを最後の行から読む習慣をつける(下の例を参照)
3. **解けた問題も音読する。**「この行は何をしているか」を声に出して説明できたら次へ

例題:こんな問題を解いていきます

No.	問題	学べること
012	1から100までの数字のうち、3の倍数だけを表示してください	for文・if文・剰余演算子(%)
034	社員名のリストから「田」を含む名前だけを新しいリストに集めてください	リスト・in演算子・append
058	「税抜価格を受け取り税込価格を返す関数」を定義し、3つの価格で試してください	関数定義・引数・戻り値
077	社員名と部署の辞書データから、部署ごとの人数を数えてください	辞書・集計の基本パターン

エラーメッセージの読み方(必修)

ターミナル — エラーの例

```
(.venv) PS C:\Users\trainee\fde-training> python knock_012.py
Traceback (most recent call last):
  File "knock_012.py", line 3, in <module>
    print(namae)
NameError: name 'namae' is not defined
(.venv) PS C:\Users\trainee\fde-training> _
```

▲ **画面イメージ:** エラーは**最後の行から**読む。「NameError: 'namae' という名前は定義されていない」→ 3行目で変数名を打ち間違えている、と特定できます。エラー文で検索する練習もここで身につけます。

1-3. 週末課題:CSV集計スクリプト(第2週後半)

1 課題データを確認する

配布される社員名簿 `employees.csv` を VS Code で開いて中身を確認します。コードを書く前にデータを目で見えるのは、FDE の実務でも鉄則です。

employees.csv(先頭5行)

```
社員ID,氏名,部署,役職,入社年
E001,山田太郎,営業部,課長,2015
E002,佐藤花子,人事部,メンバー,2021
E003,鈴木一郎,営業部,メンバー,2023
E004,田中美咲,情報システム部,部長,2010
```

2 集計スクリプトを書く

「部署ごとの人数を数えて、多い順に表示する」プログラムを作ります。まずは下のコードを写経して動かし、その後で1行ずつ意味を説明できるようにします。

aggregate.py

```
import csv

# 1. CSVファイルを読み込む
with open("employees.csv", encoding="utf-8") as f:
    rows = list(csv.DictReader(f))

# 2. 部署ごとの人数を辞書で数える
counts = {}
for row in rows:
    dept = row["部署"]
    counts[dept] = counts.get(dept, 0) + 1

# 3. 人数が多い順に表示する
print(f"社員数合計: {len(rows)}名")
for dept, n in sorted(counts.items(), key=lambda x: x[1], reverse=True):
    print(f"  {dept}: {n}名")
```

```
(.venv) PS C:\Users\trainee\fde-training> python aggregate.py
```

```
社員数合計: 42名
```

```
  営業部: 14名
```

```
  製造部: 10名
```

```
  情報システム部: 8名
```

```
  人事部: 6名
```

```
  経理部: 4名
```

```
(.venv) PS C:\Users\trainee\fde-training> _
```

▲ 画面イメージ: 集計結果が多い順に表示されれば成功。数字は配布データによって変わります。

3 発展課題に挑戦する(模倣 → 自作)

写経したコードを改造して、次の3つを自力で実装します。分からない部分は100本ノックの類題に戻って確認しましょう。

- 発展1: 「入社年が2020年以降」の社員だけを数える
- 発展2: 部署ごとに「役職が課長以上」の人数も併記する
- 発展3: 集計結果を画面表示ではなく `result.csv` に書き出す

🔧 つまづきポイントと対処

症状	原因と対処
'python' は認識されません	Python未インストール、またはPATH未設定。python.orgのインストーラーで「Add python.exe to PATH」に必ずチェックを入れて再インストール
UnicodeDecodeError が出る	CSVの文字コード問題。 <code>encoding="utf-8"</code> を <code>encoding="cp932"</code> (Excel保存のCSVの場合)に変えて試す
IndentationError が出る	行頭の空白(インデント)のズレ。Pythonは字下げが文法の一部。スペース4つで統一する
実行しても何も表示されない	<code>print()</code> を書き忘れているか、for文の中に入っていない。処理の途中に <code>print()</code> を差し込んで動きを確認する

第2章 Git・GitHub・ターミナル操作

チーム開発の共通言語。「ブランチ→コミット→プルリクエスト→レビュー対応→マージ」の一連の流れを、この週は毎日1回転させて体で覚えます。

🎯 この章の学習ゴール

- ブランチを切って作業し、意味の通るメッセージでコミットできる
- プルリクエスト(PR)を作成し、レビュー指摘に修正で応えてマージまで完走できる
- `git status` / `git diff` / `git log` で「今リポジトリがどんな状態か」を自分で確認できる

2-1. 初期設定と教材リポジトリの取得

1 Git の初期設定をする(最初の1回だけ)

コミットに記録される自分の名前とメールアドレスを設定します。

```
git config --global user.name "Taro Yamada"  
git config --global user.email "t.yamada@example.co.jp"
```

2 教材リポジトリをクローン(複製)する

GitHub 上の教材リポジトリを自分のPCにコピーします。これをクローンと呼びます。

```
git clone https://github.com/koheiashikari/company_inner_search_app2.git  
cd company_inner_search_app2
```



```
(.venv) PS C:\Users\trainee\fde-training> git clone https://github.com/koheiashikari/company_inner_search_app2.git
Cloning into 'company_inner_search_app2'...
remote: Enumerating objects: 128, done.
remote: Counting objects: 100% (128/128), done.
Receiving objects: 100% (128/128), 1.24 MiB | 4.02 MiB/s, done.
(.venv) PS C:\Users\trainee\fde-training> cd company_inner_search_app2
(.venv) PS C:\Users\trainee\fde-training\company_inner_search_app2> _
```

▲ 画面イメージ: クローン完了。フォルダに移動して以降の作業を行います。

2-2. 毎日回す「開発の1サイクル」

今週の課題は「READMEに自己紹介を追記する」「コメントの誤字を直す」など小さなものですが、流れは実務とまったく同じです。以下のサイクルを毎日1回転します。

1 作業用ブランチを作る

本体(mainブランチ)を直接触らず、作業用の「枝」を作ってそこで作業します。ブランチ名は「何をするための枝か」が分かる名前にします。

```
git switch -c feature/add-profile-yamada
```

2 ファイルを編集し、変更状態を確認する

VS Code でファイルを編集して保存したら、コミットの前に必ず `git status` (何が変わったか)と `git diff` (どう変わったか)で確認します。

```
git status
git diff
```

```
(.venv) PS ...\.company_inner_search_app2> git status
```

```
On branch feature/add-profile-yamada
```

```
Changes not staged for commit:
```

```
    modified:   README.md
```

```
(.venv) PS ...\.company_inner_search_app2> git diff
```

```
--- a/README.md
```

```
+++ b/README.md
```

```
@@ -1,2 +1,4 @@
```

```
 # company_inner_search_app2
```

```
+
```

```
+## 研修生自己紹介: 山田太郎(2026年度 未経験者コース)
```

```
(.venv) PS ...\.company_inner_search_app2> _
```

▲ **画面イメージ:** `git diff` では追加した行が **+付きの緑色** で表示されます。「自分が何を変えたか」を提出前に自分の目で確認する習慣が重要です。

3 コミットして GitHub にプッシュする

変更を「ステージング(コミット対象に載せる)」してからコミットします。メッセージは「何をなぜ変えたか」が伝わる日本語でOKです。

```
git add README.md
git commit -m "READMEに研修生の自己紹介欄を追加"
git push -u origin feature/add-profile-yamada
```

4 GitHub でプルリクエスト(PR)を作る

ブラウザで GitHub のリポジトリを開くと「Compare & pull request」ボタンが表示されるので、クリックしてPR作成画面へ。タイトルと説明(何を・なぜ・確認方法)を書いて「Create pull request」を押します。

https://github.com/koheiashikari/company_inner_search_app2/compare/main...feature/add-profile-yamada



koheiashikari / company_inner_search_app2

Open a pull request

base: main ← compare: feature/add-profile-yamada ✓ Able to merge

READMEに研修生の自己紹介欄を追加

何を変えたか

README.md に研修生の自己紹介セクションを追加しました。

確認方法

README.md をプレビュー表示し、見出しが正しく表示されることを確認済みです。

Create pull request

▲ **画面イメージ:** PR作成画面。説明欄に「何を変えたか」「確認方法」を書くのがチームのルール。レビュアーの時間を節約するのも仕事のうちです。

5 レビュー指摘に対応し、マージする

メンターがコードを確認し、コメントで指摘が入ります。**指摘はコードへの指摘であって人格への指摘ではありません。**修正して再度 push すると、同じPRに自動で反映されます。承認(Approve)が出たら「Merge pull request」ボタンでマージして完了です。

https://github.com/koheiashikari/company_inner_search_app2/pull/12 koheiashikari / company_inner_search_app2

READMEに研修生の自己紹介欄を追加 #12

Open yamada-taro が main へのマージを希望しています mentor-sato さんがレビューしました(変更を依頼)

@@ README.md @@

+## 研修生自己紹介: 山田太郎(2026年度 未経験者コース)

mentor-sato • 10 minutes ago

見出しレベルは他のセクションに合わせて `###` にしましょう。あと入社月も入れると名簿として使えます 👍 mentor-sato さんが承認しました(2回目のレビュー)

mentor-sato • 2 minutes ago

修正確認しました。LGTM!(Looks Good To Me = 問題ありません)

Merge pull request

▲ **画面イメージ:** レビューの往復の実例。1回目で指摘 → 修正して push → 2回目で承認、が典型的な流れです。

確認ポイント(第3週修了の条件)

- ・ ブランチ作成 → コミット → PR → レビュー対応 → マージ、を**1人で5回以上**完走した
- ・ コミットメッセージを見れば、コードを見なくても変更内容が想像できる
- ・ 「今どのブランチにいるか」を `git status` で常に確認する癖がついた

つまづきポイントと対処

症状	原因と対処
mainブランチで直接編集してしまった	コミット前なら <code>git stash</code> で変更を退避 → ブランチを切ってから <code>git stash pop</code> で復元
<code>push</code> が <code>rejected</code> される	リモートに自分の知らない更新がある。 <code>git pull origin ブランチ名</code> で取り込んでから再push
コミットメッセージを打ち間違えた	直前のものなら <code>git commit --amend -m "正しいメッセージ"</code> で修正できる(push前に限る)

第3章 SQLとデータモデリング基礎

FDEの仕事は「データに始まりデータに終わる」。教材リポジトリの data/ を元にしたサンプルDBを使い、データを取り出す言語=SQLを習得します。

🎯 この章の学習ゴール

- SELECT / WHERE / JOIN / GROUP BY を使った問い合わせが自力で書ける
- 「社員と部署を別テーブルに分けてIDでつなぐ」理由(正規化)を自分の言葉で説明できる
- 日本語の質問(例: 「部署ごとの平均勤続年数は?」)をSQLに翻訳できる

3-1. サンプルDBの準備

1 SQLite でデータベースを開く

研修ではインストール不要で使える **SQLite** を使います。配布された `training.db` (教材リポジトリの data/ フォルダの社員・部署・顧客データをDB化したもの)を開き、テーブルの一覧を確認します。

```
sqlite3 training.db
.tables          -- テーブル一覧を表示
.headers on      -- 結果に列名を表示する設定
.mode column     -- 結果を表形式で表示する設定
```

ターミナル — SQLite

```
PS C:\Users\trainee\fde-training> sqlite3 training.db
SQLite version 3.45.1
Enter ".help" for usage hints.
sqlite> .tables
customers  departments  employees  services
sqlite> _
```

▲ **画面イメージ**: 4つのテーブル(顧客・部署・社員・サービス)が入っています。プロンプトが `sqlite>` に変わったらSQLを直接入力できます。

2 テーブルの構造(データモデル)を理解する

社員テーブルには部署の「名前」ではなく「部署ID」だけが入っています。部署の情報は部署テーブルに1回だけ書く——これが**正規化**(1つの事実は1か所にだけ書く)です。部署名が変わっても1か所直せば済みます。

employees(社員)	departments(部署)	つながり
emp_id 社員ID(主キー) name 氏名 dept_id 部署ID hire_year 入社年 title 役職	dept_id 部署ID(主キー) dept_name 部署名 floor フロア	社員の dept_id と部 署の dept_id が同じ値同士で 結びつく(これを JOIN でつなぐ)

3-2. 問い合わせ演習(基本 → 応用)

1 基本:絞り込んで取り出す(SELECT / WHERE)

「入社が2023年以降の社員の氏名と入社年」を取り出します。

```
SELECT name, hire_year
FROM employees
WHERE hire_year >= 2023;
```

2 応用:テーブルをつないで集計する(JOIN / GROUP BY)

「部署ごとの社員数を多い順に」。2つのテーブルを **JOIN** でつなぎ、**GROUP BY** でまとめて数えます。

```
SELECT d.dept_name, COUNT(*) AS members
FROM employees e
JOIN departments d ON e.dept_id = d.dept_id
GROUP BY d.dept_name
ORDER BY members DESC;
```

```
sqlite> SELECT d.dept_name, COUNT(*) AS members
...> FROM employees e JOIN departments d ON e.dept_id = d.dept_id
...> GROUP BY d.dept_name ORDER BY members DESC;
dept_name      members
-----
営業部         14
製造部         10
情報システム部  8
人事部         6
経理部         4
sqlite> _
```

▲ **画面イメージ:** 第1章でPythonで書いた集計と同じ結果を、SQLなら3行で取り出せます。「どちらの道具をいつ使うか」の感覚もここで養います。

3 演習課題(自力で書く)

次の日本語の質問をSQLに翻訳してください。答え合わせはメンターとのペア作業で行います。

1. 「営業部の社員一覧を、入社年の古い順に」(JOIN + WHERE + ORDER BY)
2. 「入社3年以内(2024年以降)で営業部の社員の氏名」(条件の組み合わせ)
3. 「部署ごとの平均勤続年数」(2026 - hire_year の平均。AVGと計算式)
4. 「顧客ごとの契約サービス数」(customers と services のJOIN)
5. **発展:** 「社員が1人もいない部署」(LEFT JOIN と IS NULL)

✓ 確認ポイント(第4週修了の条件)

- 演習課題1〜4を、ヒントなしで正しい結果が出るまで書けた
- 「なぜ社員テーブルに部署名を直接書かないのか」を後輩に説明するつもりで話せた
- 間違ったSQLを書いたとき、結果の行数や中身から「何がおかしいか」を推測できた

第4章 Webアプリ基礎(Streamlit)

「自分の作ったものが画面で動く」体験をつくる2週間。HTTP と API の全体像を学んだうえで、Streamlit を使って社内FAQアプリをゼロから作ります。

🎯 この章の学習ゴール

- 「ブラウザとサーバー」の関係と、リクエストレスポンスの流れを図で説明できる
- Streamlit の基本部品(タイトル・入力欄・ボタン・表)で画面が作れる
- CSVデータを読み込んで検索できる「社内FAQアプリ」を完成させ、同期にデモできる

4-1. Streamlit を動かしてみる(第5週前半)

1 インストールして Hello アプリを作る

```
pip install streamlit pandas
```

app_hello.py

```
import streamlit as st

st.title("はじめての Streamlit")
name = st.text_input("あなたの名前は?")
if st.button("あいさつする"):
    st.success(f"こんにちは、{name} さん!")
```

2 アプリを起動する

`python` ではなく `streamlit run` で起動します。ブラウザが自動で開き、`http://localhost:8501` でアプリが表示されます。**localhost** とは「自分のPC自身」のこと。今は自分のPCがサーバー役をしています。

```
streamlit run app_hello.py
```


4-2. 週末課題:社内FAQアプリを作る(第5週後半～第6週)

1 FAQデータを準備する

faq.csv(抜粋)

カテゴリ, 質問, 回答
経費, 交通費の精算期限はいつですか?, 利用月の翌月10日までに精算システムで申請してください。
勤怠, 有給休暇の申請は何日前までですか?, 原則3営業日前までに上長へ申請してください。
IT, 社内Wi-Fiのパスワードを忘れました, 情報システム部のポータルから再発行申請ができます。
総務, 会議室の予約方法を教えてください, グループウェアの「施設予約」から予約できます。

2 アプリ本体を実装する

まず下のコードを写経して動かし、その後「カテゴリの絞り込み」を自力で追加します。

app_faq.py

```
import streamlit as st
import pandas as pd

st.title("💬 社内FAQ検索")
st.caption("よくある質問をキーワードで検索できます")

# 1. FAQデータを読み込む
df = pd.read_csv("faq.csv")

# 2. 検索キーワードの入力欄
keyword = st.text_input("キーワードを入力", placeholder="例: 交通費")

# 3. キーワードで絞り込んで表示
if keyword:
    hit = df[df["質問"].str.contains(keyword) | df["回答"].str.contains(keyword)]
    st.write(f"検索結果: {len(hit)} 件")
    for _, row in hit.iterrows():
        with st.expander(f"【{row['カテゴリ']}】{row['質問']}"):
            st.write(row["回答"])
else:
    st.dataframe(df)
```

[←](#) [→](#) [🔄](#)

社内FAQ検索

よくある質問をキーワードで検索できます

キーワードを入力

検索結果: 2 件

▼ 【経費】 交通費の精算期限はいつですか？

利用月の翌月10日までに精算システムで申請してください。

▶ 【経費】 出張時の交通費に上限はありますか？

▲ **画面イメージ:** 完成した社内FAQアプリ。キーワード「交通費」で2件がヒットし、クリックで回答が開閉します。第6週末に同期の前で3分デモを行います。

3 発展課題(第6週)

- **発展1:** カテゴリのプルダウン(`st.selectbox`)で絞り込みを追加
- **発展2:** ヒット0件のときに「見つかりませんでした」と案内を表示(`st.warning`)
- **発展3:** 「よく見られている質問トップ3」を集計して先頭に表示(第1章の集計の応用)
- 完成したら第2章の手順どおり **ブランチを切ってPRで提出**(復習を兼ねる)

🔧 つまづきポイントと対処

症状	原因と対処
ブラウザに「接続できません」と出る	アプリが起動していない。ターミナルで <code>streamlit run</code> が動いているか、エラーで止まっていないかを確認
コードを直したのに画面が変わらない	ブラウザ右上の「Rerun」をクリック(または画面上で R キー)。保存(Ctrl+S)を忘れているかも確認
<code>KeyError: '質問'</code>	CSVの列名とコード内の列名が一致していない。全角/半角や余分なスペースに注意

第5章 生成AI・RAG入門

いよいよ教材アプリ(社内情報検索アプリ)の世界へ。LLMをプログラムから呼び出し、RAGの仕組みを理解し、実際にアプリを起動・改修してPRを出すところまで進みます。

🎯 この章の学習ゴール

- OpenAI API をプログラムから呼び出し、プロンプトの違いによる回答の変化を実験・報告できる
- 教材アプリの起動時処理(initialize.py)の流れを図解して人に説明できる
- 教材アプリに小さな機能改修を加え、PRとして提出しマージまで完走できる

5-1. LLMをプログラムから呼び出す(第7週)

1 APIキーを安全に管理する

研修事務局から配布される OpenAI の APIキー(利用のための合鍵)を、**コードに直接書かず** `.env` ファイルに保存します。`.env` は Git にコミットしない約束のファイルです(`.gitignore` に登録済み)。

`.env`

```
OPENAI_API_KEY=sk-xxxxxxxxxxxxxxxxxxxxxxxxxxxx
```

⚠️ APIキーは絶対にコミット・チャット共有しない。漏えいすると第三者に不正利用され、高額請求につながります。誤ってコミットした場合は即メンターへ報告(キーの無効化が必要)。

2 最小の呼び出しスクリプトを書く

ask_llm.py

```
import os
from dotenv import load_dotenv
from openai import OpenAI

load_dotenv()                                # .env から環境変数を読み込む
client = OpenAI()                             # APIキーは環境変数から自動で使われる

response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {"role": "system", "content": "あなたは新人研修の講師です。専門用語を使わず説明して"},
        {"role": "user", "content": "RAGとは何ですか?3行で教えてください。"},
    ],
)
print(response.choices[0].message.content)
```

ターミナル — LLM呼び出し

```
(.venv) PS C:\Users\trainee\fde-training> python ask_llm.py
RAGは、AIが答える前にまず関係する資料を探してくる仕組みです。
見つけた資料を根拠にして回答を作るので、でたらめな答えが減ります。
社内文書の検索アプリなど、会社の中の情報を扱うときによく使われます。
(.venv) PS C:\Users\trainee\fde-training> _
```

▲ **画面イメージ:** プログラムからAIを呼び出した状態。ChatGPTの画面で使うのと同じAIですが、これで「自分のアプリにAIを組み込む」ことができるようになりました。

3 プロンプト比較実験(レポート課題)

同じ質問に対して system プロンプト (AIへの役割指示) を3パターン変えて実行し、回答の違いを表にまとめて提出します。

- パターンA: 役割指示なし
- パターンB: 「小学生にも分かるように」
- パターンC: 「箇条書き3点、各30字以内で。根拠がない場合は『分かりません』と答えること」

レポートには「業務で使うならどのパターンをなぜ選ぶか」の考察を必ず含めます。

5-2. 教材アプリの読解(第8週)

教材アプリの起動時処理 `initialize.py` を読み、「起動時に何がどの順で起きるか」をA4一枚に図解して発表します。読解の観点は次のとおりです。

読解の問い	見るべき場所
検索対象の文書はどこから読み込まれる?	<code>initialize.py</code> のデータ読み込み処理と <code>constants.py</code> のフォルダ設定
文書はどんな単位に分割される?	チャンク分割の設定値(チャンクサイズ/オーバーラップ)
「意味で検索できる形」はどこで作られる?	Embedding の生成とベクトルストア(検索用データベース)への格納
会話の記憶はどう保持される?	<code>st.session_state</code> (画面を再描画しても消えない保存場所)の初期化

5-3. 教材アプリを起動する(第9週)

1 ライブラリを一括インストールして起動する

アプリに必要なライブラリの一覧(requirements ファイル)を使って一括インストールし、起動します。

```
cd company_inner_search_app2
pip install -r requirements_windows.txt # Mac の場合: requirements_mac.txt
streamlit run main.py
```

<http://localhost:8501>

利用目的

- ☒ 社内文書検索
- ☐ 社内問い合わせ

【「社内文書検索」を選択した場合】

入力内容と関連性が高い社内文書のありかを検索できます。

社内情報検索アプリ

こちらは社内文書の検索を行うアプリです。サイドバーで利用目的を選択し、画面下部のチャット欄から質問を入力してください。



人事部に所属している社員を教えてください



人事部には6名の社員が所属しています。主なメンバーは佐藤花子(メンバー、2021年入社)、高橋健(課長、2016年入社)...(以下略)

参照元: data/社員について/社員名簿.csv

こちらからメッセージを送信できます。

▲ **画面イメージ:** 教材アプリの起動画面。左のサイドバーで利用目的を切り替え、下のチャット欄から質問します。回答の下に「参照元」の文書名が表示される点に注目——これがRAGの「根拠を示す」機能です。

✓ 確認ポイント(起動確認)

- ・「社員について教えてください」など3種類の質問をして、それぞれ参照元付きの回答が返る
- ・わざと社内文書にない質問(例:「今日の天気は?」)をして、アプリがどう振る舞うか観察した
- ・ターミナル側に出るログ(処理の記録)を眺め、1回の質問で何が起きているか推測した

5-4. 改修課題:回答に参照元ページ番号を表示する(第9～10週)

1 現状を観察し、Issue を起票する

現在はPDF文書の回答で「ファイル名」しか表示されません。「何ページ目か」も出せば利用者が原文を探しやすくなります。この改善提案を GitHub の Issue に書きます(タイトル例:「回答の参照元にページ番号を表示したい」)。

2 アタリをつける → 直す → 確かめる

改修の基本は「①どこを直すかアタリをつける → ②最小限直す → ③動かして確かめる」の繰り返しです。

- **アタリ**:参照元の表示は画面系の `components.py` にある。「参照元」「source」などの語で検索(VS Code の Ctrl+Shift+F)
- **調査**:検索結果の文書データが持つ付加情報(メタデータ)に `page` が含まれているか、`print()` を仕込んで確認
- **修正**:ファイル名の後ろに「(〇ページ)」を付けて表示するよう1～数行変更
- **確認**:PDFに関する質問を3つ試し、ページ番号が正しく出ることを目視確認

3 PRを提出してマージまで完走する

第2章の手順どおり、ブランチ → コミット → PR。PRの説明には**変更前後のスクリーンショット**を貼ります(百聞は一見にしかず)。レビュー指摘に対応してマージされたら、この章は修了です。

🔧 つまづきポイントと対処

症状	原因と対処
<code>AuthenticationError</code> (401)	APIキーの設定ミス。 <code>.env</code> のキー名のつづり、値の前後の余分なスペースを確認
ライブラリのインストールに失敗する	仮想環境が有効か確認(先頭に <code>(.venv)</code>)。エラー文の最後の行をメンターに共有
アプリの起動が異常に遅い	初回はEmbedding作成に時間がかかるのが正常。2回目以降も遅い場合は <code>data/</code> の容量とログを確認
直したのに画面が変わらない	別ファイルの同名処理を直している可能性。 <code>print()</code> で自分の修正箇所が通っているか確認

第6章 コンサルティング基礎

技術を「顧客の価値」に変換するもう半分の武器。課題の構造化 → ヒアリング → 要件定義 → 経営層向けプレゼン、と実務の順番どおりに演習します。

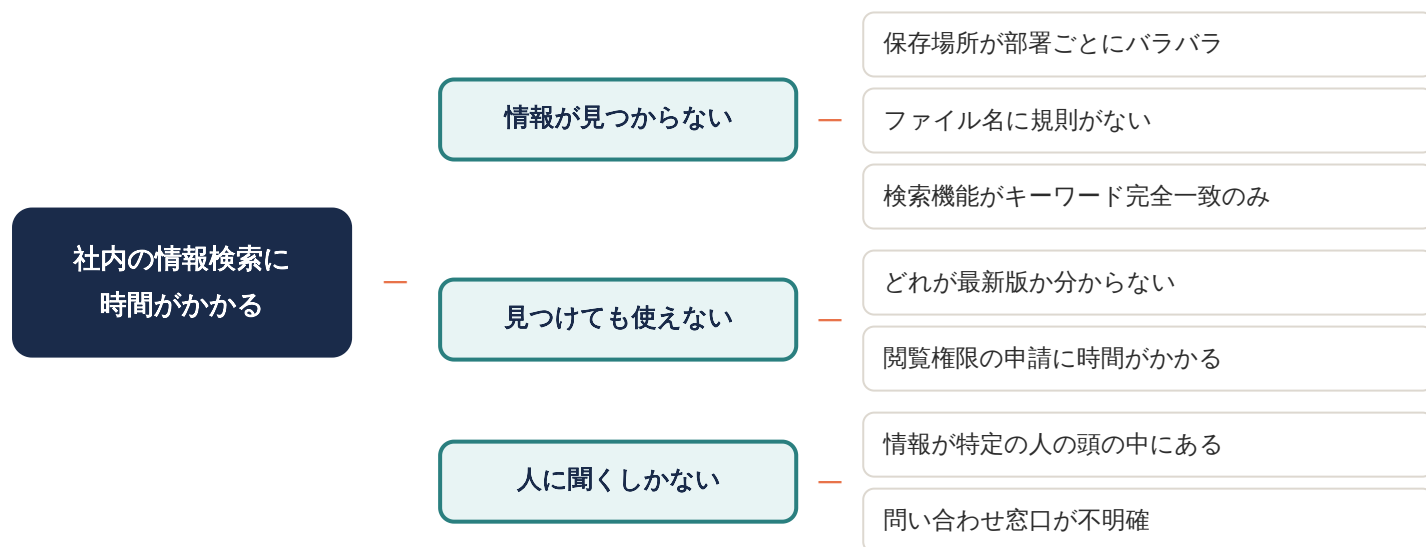
🎯 この章の学習ゴール

- 曖昧な課題をロジックツリーで「モレなくダブリなく」分解できる
- ヒアリングを設計・実施し、議事録と要件定義書に落とせる
- 技術を知らない経営層に5枚のスライドで提案を伝えられる

6-1. 課題の構造化(第11週)

1 お題:「社内の情報検索に時間がかかる」を分解する

いきなり解決策(「検索アプリを作ろう!」)に飛びつかず、まず**原因の候補を構造化**します。「なぜ時間がかかるのか?」を大きな枝(中分類)に分け、それぞれをさらに具体的な原因に分けます。各枝がMECE(モレなくダブリなく)になっているか、隣同士で意味が重なっていないかを確認します。



▲ **画面イメージ:** ロジックツリーの完成イメージ。「どの枝が一番インパクトが大きいのか」「どの枝ならデータとソフトウェアで解けるか」を議論する土台になります。

2 ドキュメント化して相互レビュー

ツリーをドキュメントにまとめ、同期同士で相互レビューします。レビュー観点:①枝はMECEか ②「原因」と「解決策」が混ざっていないか ③各末端の枝は「検証可能」か(データや聞き取りで確かめられるか)。

6-2. ヒアリングと要件定義(第12週)

1 ヒアリングを「設計」する

ヒアリングは出たところ勝負ではなく準備が9割。事前に「仮説」と「確認したいこと」を書き出した準備シートを作ります。

項目	記入例
相手の立場・関心	人事部長。採用資料作成の工数削減に関心。ITには詳しくない
仮説	過去の資料が見つからず、毎回ゼロから作り直しているのでは？
確認したい事実	資料作成の頻度/1回あたりの検索時間/どこに保存しているか
聞く順番	現状の業務の流れ → 困りごと → 理想の状態 → 制約(予算・期限)

2 ロールプレイ実施 → 議事録を24時間以内に出す

先輩社員が顧客役の30分ロールプレイを実施。終了後24時間以内に議事録を提出します。議事録の鉄則は「決まったこと」「宿題(誰が・何を・いつまでに)」「次回の予定」を必ず書くこと。発言の書き起こしではありません。

3 要件定義書にまとめて顧客役の承認を得る

ヒアリング結果を要件定義書に落とします。下の様式を使い、「やらないこと」を必ず明記するのがポイントです(スコープの合意)。

要件定義書(研修様式)

案件名:人事部 社内文書検索の効率化 | 作成:山田太郎 | 承認:顧客役・メンター | 2026/xx/xx

1. 背景・課題	採用資料の作成時に過去資料が見つからず、1回あたり平均40分の検索時間が発生している(ヒアリングで確認済み)。
2. 目的(成功基準)	資料検索の時間を平均40分 → 5分以内に短縮する。効果は導入1ヶ月後にアンケートで測定する。
3. やること(スコープ)	①人事部の共有フォルダ内文書を対象にした検索アプリの構築 ②検索結果に文書名・更新日・ページ番号を表示 ③人事部15名への説明会の実施
4. やらないこと	①他部署の文書は対象外(次フェーズで検討) ②文書の新規作成支援機能 ③スマートフォン対応
5. 前提・制約	予算:研修範囲内 / 納期:2週間 / 個人情報を含む文書は対象から除外する
6. 体制・スケジュール	開発:山田 / レビュー:メンター佐藤 / 週次で人事部長に進捗デモ(毎週金曜15:00)

▲ **画面イメージ:** 要件定義書の完成イメージ。特に「4. やらないこと」が書けているかが評価ポイント。ここが曖昧だと後から作業が際限なく増えます(スコープクリープ)。

6-3. 資料作成とプレゼン(第13週)

要件定義の内容を経営層向けの5枚スライドにまとめて発表します。経営層は技術の詳細ではなく「いくらかけて、何がどう良くなるのか」を知りたい——この視点の切り替えが本週のテーマです。

スライド	タイトル	書く内容
1枚目	結論(サマリー)	「検索時間を1/8にする社内検索アプリを2週間で導入します」— 結論を最初に言い切る
2枚目	課題	現状の困りごとを数字で示す(検索に平均40分 × 月30回 = 月20時間の損失)
3枚目	解決策	何を作るか。画面イメージ1枚+「AIが文書を根拠付きで答える」の一言。技術用語は使わない
4枚目	効果と費用	期待効果(時間換算・金額換算)と、かかる費用・リスク・その対策
5枚目	スケジュールと依頼事項	2週間の計画と、経営層に「決めてほしいこと」を明確に書く

✓ 発表の評価観点

- 専門用語(RAG、Embedding等)を一度も使わずに説明しきれたか
- 質疑で「分からないことを正直に分からないと言い、持ち帰り方を提示」できたか
- 5分の持ち時間を守れたか(実務では時間超過=準備不足と見なされます)

第7章 総合演習

16週間の集大成。架空の顧客に対し、ヒアリングから実装・デモ・プレゼンまでを一人で完走します。ここでの成果物がそのまま修了判定の材料になります。

🎯 演習課題

「架空の顧客(例:製造業の人事部)の社内文書検索を効率化せよ」

- 顧客役(先輩FDE)は業務上の困りごとは話すが、**作ってほしいものは言わない**。要件はヒアリングで自分が引き出す
- 実装は教材アプリをベースにした機能追加(例:部署別の検索フィルタ、CSV社員名簿の横断検索)

7-1. 3週間のスケジュール

週	やること	提出物(締切)
第14週	ヒアリング(2回まで) → 課題の構造化 → 要件定義	ロジックツリー(水曜)/ 要件定義書・顧客役の承認(金曜)
第15週	実装スプリント	中間デモ(水曜)/ 動く機能一式のPR(金曜)
第16週	仕上げ・リハーサル → 最終発表	最終デモ+プレゼン15分・質疑10分(金曜)/ 振り返りシート(翌月曜)

💡 週次のメンター壁打ち(毎週火曜30分)は必ず使うこと。「順調です」で終わらせず、一番不安な点を1つ持ち込むのがルールです。

7-2. 成果物チェックリスト

成果物	合格の目安
① ロジックツリー	枝がMECEで、「データとソフトウェアで解ける枝」を根拠付きで選定している
② 要件定義書	成功基準が数値で書かれ、「やらないこと」が明記され、顧客役の承認を得ている

③ 実装(PR)	要件定義書のスコープと一致し、レビュー指摘対応まで完了。動作確認の手順がPRに書かれている
④ 最終デモ・プレゼン	実物のアプリで顧客の業務シナリオを再現。質疑で誠実に応答できる
⑤ 振り返りシート	うまくいったこと/いかなかったこと/次に変えることが具体的に書かれている

7-3. 評価ルーブリック(抜粋)

軸	レベル3(修了ライン)	レベル4～5(優秀)
技術	レビュー指摘が軽微な修正のみでPRをマージできる	代替案を比較して設計判断を説明できる/テストを自発的に書いている
コンサル	ヒアリングから要件定義書を作り、顧客役の承認を得られる	顧客役の想定外の要求に対し、スコープ交渉で着地させられる
ドメイン	顧客(人事部)の業務の流れを踏まえた提案ができる	業務指標(工数・コスト)で効果を定量化して語れる

✓ 修了判定(カリキュラム共通基準の再掲)

- 顧客役へのヒアリングから要件定義書を作成できる
- 教材アプリ相当のRAGアプリを自力で改修・機能追加できる
- 成果物を顧客役の前でデモ・プレゼンし、質疑に答えられる

修了者は配属後3ヶ月間、月1回のフォローアップ会に参加します。本書と自分の成果物(ポートフォリオ)は配属先での最初の武器になります。